mygit Agent

Days 31–35 — Natural Language Git Control

Day	What you build	Key concept
31	Tool definitions + ADK agent scaffold	Tool contract design
32	Agent loop + basic commands working	Orchestration
33	Ambiguity handling + error recovery	Agentic reasoning
34	Commit message generation from diff	LLM reads your code
35	Ollama offline support + demo	Local vs cloud models

"The agent is only Day 31 because the CLI is Day 30. Build the tools first, make them reliable, then add the intelligence on top. That sequencing is the engineering lesson."

What the user experiences

```
$ mygit agent
mygit-agent> stage everything and commit saying i fixed the login bug

[checking status...]
[staging all files...]
[committing...]
Committed 3 files with message "fix login bug".  Push now?

mygit-agent> yes

[pushing to origin/main...]
Done.  Pushed 1 commit to origin/main.

mygit-agent> get my feature branch up to date with main

[checking for uncommitted changes...]
[fetching from origin...]
[rebasing onto main...]
Done.  feature is now 3 commits ahead of main.
```

Prerequisite: mygit 30-day plan complete and deployed. Every CLI command returns clean, parseable output. **Model:** Gemini 2.0 Flash (free) or any Ollama model (offline).

How the Agent Works

Read this before Day 31.

The agentic loop — four steps, repeated:

1. **Understand.** The LLM reads the user's message + the list of available tools + the conversation history.
2. **Decide.** The LLM outputs a structured tool call: `{name: "git_add", args: {path: "."}}`.
3. **Execute.** Your code runs the tool, captures the output.
4. **Observe.** The output is added to the conversation. The LLM reads it and decides the next step — call another tool, or respond to the user.

This loop continues until the LLM decides the task is complete and responds in natural language. ADK manages this loop for you. Your job is to define the tools and write the system prompt.

The most important design decision: tool descriptions.

The model never reads your code. It only reads your tool descriptions to decide whether to call a tool and what arguments to pass. A vague description produces wrong tool calls. A precise description produces correct ones.

Bad: `"Adds files."`
Good: `"Stage files for the next commit. Pass '.' to stage all changed files in the working directory, or a specific path to stage one file. Run this before git_commit. Returns a list of staged files."`

Every tool description should answer: what does this do, when should I call it, what arguments does it take, what does it return.

Cloud vs local models — the trade-off:

	Cloud (Gemini / GPT-4o)	Local (Ollama)
Setup	API key only	Install Ollama + download model (2–5 GB)
Internet	Always required	Only for initial download
Tool calling	Very reliable	Varies by model
Best local model	—	Qwen2.5-Coder 7B (code tasks)
Speed	Fast	Depends on hardware
Privacy	Prompts sent externally	Everything stays local

Recommendation: build with Gemini first. Add Ollama as a config switch on Day 35. One environment variable swap — no code changes in the agent logic.

The interview connection:

You are using Google ADK at Wells Fargo professionally. You are using the same framework here to extend a system you built yourself. The interview answer writes itself:

"I applied the same agentic patterns I use at Wells Fargo to my own git implementation. The agent wraps my CLI as a set of tools. I learned that tool contract design is the real engineering challenge — the LLM reasoning is handled by the model, but whether the agent calls the right tool reliably depends entirely on how well you've described what each tool does."

Days 31–35 — The Agent

5 sessions, 1.5–2 hrs each

☐ Day 31 Tool definitions + ADK scaffold *1.5 hrs*

Create `src/agent/tools.ts`. Define one function per mygit command. Each function: calls `runMygit([...args])`, returns the stdout as a string. Write the description precisely (see the design principle on the previous page). Tools to define today: `git_status`, `git_add(path)`, `git_commit(message)`, `git_push(remote, branch)`, `git_diff()`, `git_diff_staged()`, `git_log(n)`, `git_branch(name?)`, `git_checkout(branch)`. Then scaffold the ADK agent: import tools, write the system prompt (instructions below), wire up the `readline` input loop that passes user messages to the agent. Do not worry about responses yet — just get it to run without crashing.

Test: `mygit agent` starts without errors. Typing a message and pressing Enter does not crash. The agent may not respond intelligently yet — that is fine.

System prompt to use on Day 31:

```
You are a git assistant controlling a mygit repository.
When the user gives you a task, use the available tools to complete it step by step.
Rules:
- Always run git_status before staging or committing.
- Always confirm with the user before pushing.
- If the user's request is ambiguous, ask one clarifying question before acting.
- When a tool returns an error, explain what went wrong and ask how to proceed.
- Keep responses concise.  Show the important output, not all of it.
- Never guess at file paths.  If unsure which files to stage, ask.
```

☐ Day 32 Agent loop + basic commands working *1.5 hrs*

Wire the tools into the ADK agent. Register each tool function. Run the agent loop. Test these five phrases and verify each produces the correct sequence of tool calls:

1. "what is the current status" → calls `git_status`, shows output.
2. "stage all files" → calls `git_status`, then `git_add('.')`.
3. "commit with message hello world" → calls `git_commit('hello world')`.
4. "show me recent commits" → calls `git_log(5)`.
5. "add . and commit saying fix bug and push" → three tool calls in sequence, asks before pushing.

Test: All five phrases trigger the correct tool calls in the correct order. The push confirmation works — agent asks before pushing and waits for yes/no.

☐ Day 33 Ambiguity handling + error recovery *1.5 hrs*

Test and fix three failure modes:

Ambiguous staging. Type "commit my changes." If there are unstaged files, the agent should call `git_status` first, then ask: "You have 3 unstaged files. Stage all of them, or just commit the 2 already staged?" Adjust your system prompt if the agent doesn't ask.

Push rejection. Manually create a non-fast-forward situation (push from elsewhere). Type "push." The agent receives the rejection error. It should say: "Push was rejected — the remote has commits you don't have. Should I fetch first?" and proceed accordingly.

Nothing staged. Type "commit." Agent calls `git_status`, sees nothing staged, tells the user instead of calling `git_commit`.

Test: All three failure modes produce helpful responses rather than crashing or silently failing. The agent never calls `git_commit` when there is nothing staged.

☐ Day 34 Commit message generation from diff *2 hrs*

Add a new tool: `git_generate_commit_message()` — calls `git_diff_staged()`, passes the output to the LLM in a separate prompt, returns a suggested commit message.

The inner prompt: `"Given this diff, write a concise, imperative commit message (50 chars max). Just the message, nothing else: {diff}"`

Wire it into the agent: when the user says "commit with a good message" or "commit and generate a message," the agent calls `git_diff_staged` to see what changed, calls `git_generate_commit_message` to get a suggestion, shows it to the user for approval, then commits.

Also add: `git_stash()` and `git_stash_pop()` tools. Teach the agent: when user says "get feature up to date with main," stash uncommitted changes, fetch, rebase, pop stash.

Test: "Commit with a good message" generates a sensible message based on the actual diff and asks for confirmation before committing. Message is imperative mood, under 60 chars.

☐ **Day 35 Ollama offline support + demo video** *2 hrs*

Add Ollama support with a single config switch.
In `.env`: `MODEL_PROVIDER=gemini` or `MODEL_PROVIDER=ollama`.
In `src/agent/client.ts`:

```
const client = process.env.MODEL_PROVIDER === 'ollama'
  ?  new OpenAI({
    baseURL: 'http://localhost:11434/v1',
    apiKey:  'ollama'
  })
  :  new GoogleGenerativeAI({ apiKey:  process.env.GEMINI_API_KEY })

const MODEL = process.env.MODEL_PROVIDER === 'ollama'
  ?  'qwen2.5-coder:7b'
  :  'gemini-2.0-flash'
```

No other code changes. The agent logic, tools, and system prompt are identical. Test with Ollama: `ollama pull qwen2.5-coder:7b`, set the env var, run the agent. Note any tool calling reliability differences between the two models. Then record the demo video (see next page).

Test: `MODEL_PROVIDER=ollama mygit agent` runs fully offline after model download. `MODEL_PROVIDER=gemini mygit agent` uses the cloud. Same behaviour, one env var.

The Demo Video

Record a 3-minute video. Show these five moments in order:

1. **Natural language commit.** Type "stage everything and commit saying I added the login feature." Show the agent calling status, add, commit in sequence.
2. **Ambiguity.** Type "commit my changes" when some files are staged and some are not. Show the agent asking which files to include before acting.
3. **Message generation.** Type "commit with a good message." Show the agent reading the diff and suggesting a message. Approve it.
4. **Error recovery.** Trigger a push rejection. Show the agent explaining the situation and offering to fetch first.
5. **Offline mode.** Switch to Ollama. Run the same basic command. Show it working without internet. Mention the trade-off in tool calling reliability.

Interview Prep — Questions You Will Be Asked

Q: How does the agent decide which tool to call?

The LLM reads the user's message alongside the descriptions of all available tools. It outputs a structured tool call — a JSON object with the function name and arguments. The model never sees the implementation, only the description. This is why tool descriptions are the most important design decision: a precise description produces reliable tool calls, a vague one produces the wrong tool or wrong arguments.

Q: What happens when a tool call fails?

The tool's output (including the error message) is returned to the LLM as the observation for that step. The LLM reads it and decides the next action — explain the error to the user, try a different tool, or ask the user how to proceed. The agent never silently swallows errors because the LLM always reads the output. This is one of the key reliability properties of the tool-calling loop.

Q: What was the hardest engineering problem?

Tool calling reliability with local models. Cloud models like Gemini almost never hallucinate a tool call — they output valid JSON with the correct function name and arguments. Smaller local models sometimes output the tool call in the wrong format, call a function that doesn't exist, or respond in natural language instead of calling a tool at all. The fix was two things: more explicit tool descriptions that include negative examples ("do not call this if there is nothing staged"), and a retry loop that detects invalid tool call format and re-prompts with a stricter instruction.

Q: Why did you support both cloud and local models?

Privacy and offline use. Cloud models send your commit messages and diffs to external servers. For proprietary codebases that is not acceptable. Local models via Ollama run entirely on the user's machine after an initial download. The implementation is one environment variable switch because Ollama exposes an OpenAI-compatible API endpoint at localhost — the agent logic, tool definitions, and system prompt are identical in both modes. The trade-off is tool calling reliability: cloud models are more reliable, local models are more private.

The one-sentence pitch with the agent included:

"I built a git implementation from scratch in TypeScript, added a RAG pipeline that indexes code on every push, and then wrapped the whole CLI in a Google ADK agent so you can control it in natural language — using the same agentic patterns I use professionally at Wells Fargo."

The full project at a glance after Day 35:

mygit CLI	Content-addressed object store, 7 commands, push/pull/clone
Backend API	Express + Prisma + PostgreSQL, auth, file browsing, commits
Frontend UI	React file browser, file viewer, commit history
RAG pipeline	Embed on push, pgvector search, GPT-4o synthesis, Ask tab
AI Agent	Google ADK, natural language control, cloud + offline modes

Five layers. One coherent system. Every layer you can explain from first principles.